Desarrollo basado en especificaciones

Gestor de incidencias con Copilot

Santiago Nicolás Blacio Quimis
Frank Emilson Cárdenas Figueroa
Universidad Técnica de Machala, 4A

El caso de la empresa

Pérdida de versiones y trabajo desorganizado
Errores en producción y poca documentación
Caso académico sin métricas de una empresa real

Proceso propuesto

Git conserva el historial y GitHub permite revisarlo
La especificación define qué debe cumplir el código
Copilot implementa y Actions repite verificaciones

Las seis fases de SDD

1 Requerimientos
2 Especificación
3 Plan técnico
4 Implementación con IA
5 Verificación
6 Evolución

Reglas del gestor de incidencias

Título obligatorio, máximo 120 caracteres
Prioridad Baja, Media o Alta y fechas UTC
Búsqueda, filtros combinados y persistencia XML

Criterio de aceptación

Dada una incidencia Pendiente
Al intentar pasar directamente a Resuelta
El sistema rechaza el cambio y conserva los datos

Arquitectura Windows Forms

Entidades representan las incidencias
Controladores validan y coordinan las reglas
Datos guarda XML y Formularios presenta la interfaz

GitHub y revisión en dúo

Main y feature/gestor-incidencias publicadas
PR #1 en borrador, revisión del compañero pendiente
Issues, Kanban y milestone propuestos por completar

Copilot en Visual Studio

Generación real mediante el chat del proyecto
Cambios por tareas y formularios con Designer.cs
SDD aplicado sin ejecutar la CLI de Spec Kit

Pruebas y automatización

Ocho grupos de aserciones aprobados en Release
El PR mostró comprobaciones opcionales superadas
Actions compila, verifica y conserva binarios

Demostración y evolución

Alta, edición, transiciones y filtros comprobados
Los datos se conservaron después de reiniciar
Próxima evolución con prueba fallida antes del código

Conclusiones

La especificación fija criterios para aceptar cambios
Git conserva el contexto y CI repite comprobaciones
La revisión humana exige comprender el código